

强化学习代码审查与数学模型分析

悬崖寻路 (Cliff Walking) 任务实现

Zhiyun Yu

2026 年 5 月 22 日

一、数学构成分析：状态与动作空间

- 状态空间 (State Space, S)

- 全局网格为 4×12 ，共定义了 48 个离散状态： $S = \{0, 1, \dots, 47\}$ 。
- 特殊状态：起点 $s_{\text{start}} = 36$ ，终点 $s_{\text{goal}} = 47$ ，悬崖状态集 $S_{\text{cliff}} \subset S$ 。
- 代码映射：对应 `environment.py` 中的 `NUM_STATES` 以及 `state_to_rc()` 二维映射函数。

- 动作空间 (Action Space, A)

- 每个状态下有 4 个离散动作： $A(s) = \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$ 。
- 代码映射：对应 `ACTION_DELTAS` 中定义的行、列坐标偏移量。

一、数学构成分析：转移概率与奖励函数

- 状态转移概率 (Transition Dynamics, P)

- 核心环境为确定性动力学: $P(s' | s, a) \in \{0, 1\}$ 。
- 悬崖非标准转移: 跌入悬崖的状态会触发环境机制, 直接重置到起点 $s' \rightarrow s_{\text{start}}$ 。
- 终点吸收态: 终点 (Goal) 为吸收状态, 满足 $P(s | s_{\text{goal}}, a) = 1$ 。
- 代码映射: 对应 `build_transitions()` 构建的形状为 (48, 4, 48) 的张量 `TRANSITIONS`。

- 奖励函数 (Reward Function, R)

- 跌入悬崖惩罚: $R(s, a) = -100$
- 到达目标奖励: $R(s, a) = 0$
- 常规移动步数惩罚: $R(s, a) = -1$
- 代码映射: 对应 `build_rewards()` 构建的形状为 (48, 4) 的矩阵 `REWARDS`。

一、数学构成分析：贝尔曼方程与策略提取

贝尔曼最优方程 (Bellman Optimality Equation)

$$V_{k+1}(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) V_k(s') \right]$$

- 代码实现: `Q = REWARDS[s] + gamma * (TRANSITIONS[s] @ V)`
- 向量化亮点: 利用 NumPy 矩阵乘法 `@` 算子完美替代了内层对 s' 的显式 `for` 循环, 大幅提升了计算性能。

策略提取 (Policy Extraction)

$$\pi^*(s) = \arg \max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) V^*(s') \right]$$

- 代码实现: `extract_policy()` 函数直接复用了上述最优 Q 值的计算逻辑, 并通过 `argmax()` 提取贪婪策略。

二、代码审查：架构设计优点 (Strengths)

- **矩阵化与向量化加速**

- 在值迭代的核心部分，使用张量乘法替代了经典理论中的三重显式嵌套循环，规避了 Python 在循环层面上的效率瓶颈。

- **预计算与空间换时间思想**

- 在 `environment.py` 模块加载时一次性计算好 REWARDS 和 TRANSITIONS 矩阵，避免了在价值迭代算法和后续仿真中进行重复的条件判断与算力浪费。

- **严谨的吸收态隔离**

- 通过 `if is_terminal(s): V_new[s] = 0.0` 显式截断了终点状态的价值累加，彻底防止了无限步长片段下的价值发散。

- **良好的软件工程工程规范**

- 模块拆分清晰、类型提示 (Type Hints) 覆盖完整，且每个子模块均包含 `if __name__ == "__main__":` 的自测逻辑。

二、代码审查：改进与优化建议 (Feedback)

- 优化价值函数的迭代收敛机制 (In-place Update)

- 当前代码采用类似 Jacobi 迭代的方式（每次开辟新数组 `V_new`）。如果改为原地更新（直接在原数组 `V` 上修改，类似 Gauss-Seidel 迭代），由于最新计算的价值能立即参与到下一个状态的更新中，通常收敛速度能提升近一倍，且更节省内存。

- 消除硬编码的“魔法数字” (Magic Numbers)

- `START_STATE` 的定义硬编码写死了基准行 `3 * NUM_COLS`。建议使用 $(\text{NUM_ROWS} - 1) * \text{NUM_COLS}$ 替代，以增强代码对未来网格尺寸变动的动态适应能力。

- 探讨片段式任务中 γ (Gamma) 默认值的取值

- 悬崖寻路属于典型的带终点片段式任务 (Episodic Task)，教材中通常设置 $\gamma = 1.0$ 来直观反映走过的纯粹步数惩罚。当前默认的 0.99 虽然能收敛，但会导致离终点较远的最优状态绝对负价值在视觉上偏小。